Recruitment Agency Database Model

Business description and logical design | Rūta Karulaitė

Business Description

Benefits and Project Vision

The agency manages vacancies for several companies, candidate applications, interviews, additional services and placements. The 15-table model connects these activities while preserving earlier facts. A person has one identity even when acting as both a candidate and a company representative.

The database supports tracing an application from submission to hiring, measuring time between stages, identifying services received, and recovering earlier job terms or interview schedules. Candidate contacts and company details are stored separately from recruitment transactions, avoiding conflicting copies.

Scope and Assumptions

Each job belongs to one company. One candidate may apply to many jobs, and each job may receive many candidates. There is one application per candidate and stable job; reconsideration adds stage events.

An application may have several interview rounds, with one company representative per round, and at most one placement. A placement requires prior approval. Panel interviews and repeat application attempts are outside this scope.

Salary figures are annual gross EUR; service fees are EUR. The named services are resume_advice, coaching and skills_testing. Each session is recorded independently, including sessions before any application.

Skills and locations are canonical atomic labels, without catalog metadata. Past employers need not be agency clients. Billing, payroll and CV file storage are excluded.

Model description

Definitions and Acronyms

Party means one person or company; candidate and representative are roles, not separate identities. A revision is a full historical snapshot. An event records a dated process change or assertion. A placement is a hiring arrangement reached through an application.

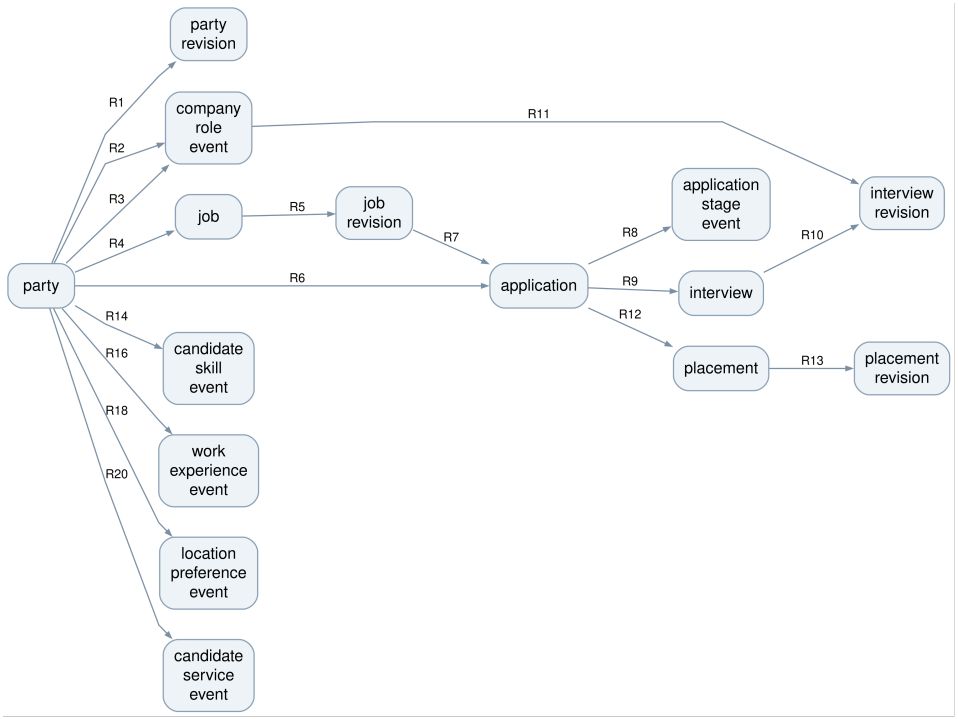
PK = primary key; FK = foreign key; NN = NOT NULL; UQ = UNIQUE; D = DEFAULT; C = row CHECK. IDENTITY denotes a generated bigint identifier. effective_at is business time; recorded_at is the time the system learned a fact. timestamptz stores an instant, not the original timezone name; date stores employment calendar dates.

Modeling Approach and 3NF

The design process separated participants, recruitment processes and changing facts; resolved the candidate/job M:N relationship with application; then separated stable identities from revisions. PK/FK directions and historical scenarios were checked last. The hardest choice was retaining every requested history within 15 tables: fixed service and stage codes use CHECK lists instead of catalogs with no required metadata.

1NF: every field is atomic and repeated skills, locations and sessions use separate rows. 2NF: changing attributes depend on the complete composite revision key; fixed ownership remains on its anchor. 3NF: candidate contacts, company names and application identity are not copied into interviews or placements. Specific dependencies and separation reasons appear under each object.

Logical Scheme



Overview of all 15 relations. R numbers match the detailed PNG, which also shows every field, constraint, cardinality and self-reference. Teal denotes anchors, blue revisions and ochre events.

Relationships and Cardinalities

application implements candidate-to-job M:N through candidate_id and the advertised job revision FK. UNIQUE(candidate_id, job_id) prevents a second submission to the same vacancy. Each revision belongs to one anchor; every anchor requires its first revision at commit. Applications require an initial submitted event, may have 0..* interviews and 0..1 placement. A company and a person may each have 0..* role events. Candidate histories have 0..* events; a predecessor and its direct successor are each optional, with at most one successor because predecessor_id is unique.

Historical Data and Integrity

All facts are append-only: normal writes must reject UPDATE, DELETE and TRUNCATE. Changing profiles, advertisements, interview arrangements and hiring terms adds revisions. Stage and representation changes add events. Skills, work experience, location preferences and service sessions use predecessor chains; withdrawals or cancellations retain prior rows.

For business time B and knowledge time K, filter effective_at <= B and recorded_at <= K, then select the greatest effective_at, recorded_at and revision/event number. Apply this within each anchor, company/person pair, or reconstructed assertion chain. Do not use only the newest row when querying history. For example, an application pinned to job revision 1 retains its original salary range after revision 2 is published.

PK/FK/UQ/NN/defaults and row checks are specified below. All FK deletions are RESTRICT and key updates are NO ACTION. The DDL must additionally validate party kinds, sequential revisions, initial child rows, chain ownership and chronology, unique candidate/skill and candidate/location roots, open advertised terms at submission, and approval before placement. The interviewer must have an active appointment for the vacancy company as known when scheduled. These cross-row rules need database procedures/triggers with parent locks; a row CHECK or FK alone is insufficient. Cancel or end a placement before a later rejection/withdrawal.

Objects

All relations are in 3NF under the stated assumptions. Field dictionaries list every attribute and constraint; examples show selected columns from the connected synthetic sample_data.json. Composite PKs and FK targets are stated explicitly.

party

A stable participant identity avoids duplicating someone who applies and later represents a company. The generated party_id determines kind and registration time; neither non-key field determines another. Names and contacts belong in revisions because they can change.

Field name	PostgreSQL type	Constraints	Field meaning
party_id	bigint	PK IDENTITY	Stable participant identifier
party_kind	text	NN CHECK	person or company
created_at	timestamptz	NN D=now()	Identity registration time

Checks: party_kind IN ('person','company').

Example data

party_id	party_kind
1	person
2	company
3	person

party_revision

Profile snapshots are separate so contact changes do not rewrite previous facts. The full PK (party_id, revision_no) determines the name, contact and times; party_id alone does not determine a historical snapshot. Stable kind stays in party. Emails are not unique because contacts may be shared or reused.

Field name	PostgreSQL type	Constraints	Field meaning
party_id	bigint	PK FK NN	Participant being described
revision_no	integer	PK NN CHECK	Positive sequential revision
effective_at	timestamptz	NN	When the snapshot takes effect
recorded_at	timestamptz	NN D=now()	When the system learned the fact
display_name	text	NN CHECK	Person full name or company name
email	text	NULL	Contact email as recorded
phone	text	NULL	Contact number as recorded

PK (party_id, revision_no); FK (party_id) → party(party_id).

Checks: revision_no > 0; btrim(display_name) <> ''.

Example data

party_id	revision_no	display_name	email
1	1	Alex Morgan	alex@example.test
1	2	Alex Morgan	alex.new@example.test
2	1	Northstar Labs	hr@example.test

company_role_event

Representation is a dated relationship: one person can represent several companies and one company can appoint several people. role_event_id determines the pair, action and times. Profile facts stay outside this table, avoiding transitive dependencies. Party-kind validation requires a trigger.

Field name	PostgreSQL type	Constraints	Field meaning
role_event_id	bigint	PK IDENTITY	Role event identifier
company_id	bigint	FK NN	Represented company
person_id	bigint	FK NN	Person acting for that company
effective_at	timestamptz	NN	Business time of the change
recorded_at	timestamptz	NN D=now()	System recording time
action	text	NN CHECK	appointed or ended

FK (company_id) → party(party_id); FK (person_id) → party(party_id).

Checks: action IN ('appointed','ended'); company_id <> person_id.

Example data

role_event_id	company_id	person_id	action
201	2	3	appointed
202	2	1	appointed

job

A stable vacancy belongs to exactly one company. job_id determines company_id and creation time; editing advertised terms cannot change ownership. Keeping company_id here, rather than on every revision, removes a partial dependency from the composite revision key.

Field name	PostgreSQL type	Constraints	Field meaning
job_id	bigint	PK IDENTITY	Stable vacancy identifier
company_id	bigint	FK NN	Client company owning the vacancy
created_at	timestamptz	NN D=now()	Vacancy registration time

FK (company_id) → party(party_id).

Example data

job_id	company_id
10	2
11	2

job_revision

Versioned advertisements preserve the terms candidates saw. The whole PK (job_id, revision_no) determines title, location, salary and status; job_id alone does not. Company ownership stays in job. Applications pin an exact revision, so a later salary edit cannot alter earlier submissions.

Field name	PostgreSQL type	Constraints	Field meaning
job_id	bigint	PK FK NN	Vacancy being revised
revision_no	integer	PK NN CHECK	Positive sequential revision
effective_at	timestamptz	NN	Business validity start
recorded_at	timestamptz	NN D=now()	System recording time
title	text	NN CHECK	Advertised job title
description	text	NN	Job duties and requirements
location_label	text	NN CHECK	One canonical work location or remote
salary_min_eur	numeric(12,2)	NULL CHECK	Optional minimum annual gross EUR
salary_max_eur	numeric(12,2)	NULL CHECK	Optional maximum annual gross EUR
status	text	NN D=open CHECK	open or closed

PK (job_id, revision_no); FK (job_id) → job(job_id).

Checks: revision_no > 0; btrim(title) <> ''; btrim(location_label) <> ''; salary_min_eur IS NULL OR salary_min_eur >= 0; salary_max_eur IS NULL OR salary_max_eur >= 0; salary_min_eur IS NULL OR salary_max_eur IS NULL OR salary_max_eur >= salary_min_eur; status IN ('open','closed').

Example data

job_id	revision_no	salary_min_eur	salary_max_eur
10	1	40000	50000
10	2	42000	52000

application

This bridge represents candidate/job M:N and anchors progress, interviews and hiring. The generated PK gives child tables compact references; the alternate candidate/job key prevents duplicate submissions. job_id does not determine the chosen revision across all applications. Current stage is derived, avoiding duplicated state.

Field name	PostgreSQL type	Constraints	Field meaning
application_id	bigint	PK IDENTITY	Submission identifier
candidate_id	bigint	FK NN	Person applying
job_id	bigint	FK NN	Stable vacancy in the chosen revision
job_revision_no	integer	FK NN	Advertised revision at submission
submitted_at	timestamptz	NN D=now()	Submission business time

UQ (candidate_id, job_id); FK (candidate_id) → party(party_id); FK (job_id, job_revision_no) → job_revision(job_id, revision_no).

Example data

application_id	candidate_id	job_id	job_revision_no
100	1	10	1
101	1	11	1

application_stage_event

Separate stage occurrences preserve review, rejection and reconsideration instead of overwriting status. The complete PK (application_id, event_no) determines stage, times and note. Candidate and job are derived from application, so no partial or transitive dependency is introduced.

Field name	PostgreSQL type	Constraints	Field meaning
application_id	bigint	PK FK NN	Application progressing through stages
event_no	integer	PK NN CHECK	Sequential event number
effective_at	timestamptz	NN	Business time of stage change
recorded_at	timestamptz	NN D=now()	System observation time
stage	text	NN CHECK	submitted review approved rejected withdrawn
note	text	NULL	Reason or reviewer explanation

PK (application_id, event_no); FK (application_id) → application(application_id).

Checks: event_no > 0; stage IN ('submitted','review','approved','rejected','withdrawn').

Example data

application_id	event_no	stage	effective_at
100	1	submitted	2026-01-10T09:00Z
100	2	review	2026-01-12T09:00Z
100	3	approved	2026-01-25T09:00Z

interview

A stable round is separate from its schedule, allowing several rounds per application and several revisions per round. interview_id determines application_id and creation time. Candidate and company are derived from the application and job rather than copied.

Field name	PostgreSQL type	Constraints	Field meaning
interview_id	bigint	PK IDENTITY	Interview round identifier
application_id	bigint	FK NN	Application being discussed
created_at	timestamptz	NN D=now()	Round registration time

FK (application_id) → application(application_id).

Example data

interview_id	application_id
300	100
301	101

interview_revision

Rescheduling, interviewer changes and feedback add snapshots. The complete revision key determines the meeting details; the fixed application link remains in interview. role_event_id supplies the representative appointment without duplicating a name or company. This prevents inconsistent interviewer/company copies.

Field name	PostgreSQL type	Constraints	Field meaning
interview_id	bigint	PK FK NN	Interview round
revision_no	integer	PK NN CHECK	Sequential revision number
effective_at	timestamptz	NN	When this revision takes effect
recorded_at	timestamptz	NN D=now()	When the change was recorded
role_event_id	bigint	FK NN	Appointment of the company interviewer
starts_at	timestamptz	NN	Scheduled meeting start
ends_at	timestamptz	NN CHECK	Scheduled meeting end
status	text	NN D=scheduled CHECK	scheduled completed cancelled
venue	text	NN	Physical address or meeting link
feedback	text	NULL	Feedback on a completed round

PK (interview_id, revision_no); FK (interview_id) → interview(interview_id); FK (role_event_id) → company_role_event(role_event_id).

Checks: revision_no > 0; ends_at > starts_at; status IN ('scheduled','completed','cancelled'); feedback IS NULL OR status = 'completed'.

Example data

interview_id	revision_no	starts_at	status
300	1	2026-01-15T10:00Z	scheduled
300	2	2026-01-16T10:00Z	scheduled

placement

A hiring arrangement is distinct from application progress. placement_id and unique application_id are candidate keys. Candidate, job and company are derived through application, avoiding transitive dependencies and multiple placements for one submission.

Field name	PostgreSQL type	Constraints	Field meaning
placement_id	bigint	PK IDENTITY	Hiring arrangement identifier
application_id	bigint	FK UQ NN	Successful application
created_at	timestamptz	NN D=now()	When the placement was registered

FK (application_id) → application(application_id).

Example data

placement_id	application_id
400	100
401	101

placement_revision

Salary, start-date changes and employment outcomes are historical snapshots. The whole PK (placement_id, revision_no) determines these terms. Fixed application identity stays in placement, and advertised salary is not copied because agreed salary is a separate fact.

Field name	PostgreSQL type	Constraints	Field meaning
placement_id	bigint	PK FK NN	Hiring arrangement
revision_no	integer	PK NN CHECK	Sequential revision number
effective_at	timestamptz	NN	When these terms take effect
recorded_at	timestamptz	NN D=now()	System recording time
start_date	date	NN	Agreed employment start
end_date	date	NULL CHECK	Actual end if employment ended
annual_salary_eur	numeric(12,2)	NN CHECK	Agreed annual gross salary in EUR
status	text	NN D=planned CHECK	planned started ended cancelled

PK (placement_id, revision_no); FK (placement_id) → placement(placement_id).

Checks: revision_no > 0; annual_salary_eur > 0; end_date IS NULL OR end_date >= start_date; status IN ('planned','started','ended','cancelled'); (status = 'ended') = (end_date IS NOT NULL).

Example data

placement_id	revision_no	start_date	status
400	1	2026-02-01	planned
400	2	2026-02-01	started

candidate_skill_event

Each row asserts or withdraws one skill level. The event PK determines the full assertion; the unique optional predecessor FK creates a linear history. Skill labels have no catalog attributes here, so no label-to-description dependency exists. A trigger prevents competing roots for one candidate/skill.

Field name	PostgreSQL type	Constraints	Field meaning
skill_event_id	bigint	PK IDENTITY	Skill assertion identifier
candidate_id	bigint	FK NN	Person whose skill is recorded
predecessor_id	bigint	FK UQ NULL	Prior assertion being superseded
effective_at	timestamptz	NN	When proficiency or withdrawal applies
recorded_at	timestamptz	NN D=now()	System recording time
skill_label	text	NN CHECK	Canonical atomic skill label
proficiency	text	NN CHECK	basic intermediate advanced expert
is_withdrawn	boolean	NN D=false	True removes the assertion from matching

FK (candidate_id) → party(party_id); FK (predecessor_id) → candidate_skill_event(skill_event_id).

Checks: btrim(skill_label) <> ''; proficiency IN ('basic','intermediate','advanced','expert'); predecessor_id IS NULL OR predecessor_id <> skill_event_id.

Example data

skill_event_id	predecessor_id	skill_label	proficiency
500	NULL	SQL	intermediate
501	500	SQL	advanced

work_experience_event

Each root describes one employment episode; successors correct it or add an end date. The event key determines its complete snapshot. Employer name is a historical label, not a unique company identifier. The self-FK preserves changes without repeating a stable episode/candidate dependency in a composite key.

Field name	PostgreSQL type	Constraints	Field meaning
experience_event_id	bigint	PK IDENTITY	Employment assertion identifier
candidate_id	bigint	FK NN	Person whose experience is described
predecessor_id	bigint	FK UQ NULL	Previous version of this episode
effective_at	timestamptz	NN	When this assertion is valid
recorded_at	timestamptz	NN D=now()	System recording time
employer_name	text	NN CHECK	Employer label supplied in the CV
job_title	text	NN CHECK	Historical position title
start_date	date	NN	Employment start
end_date	date	NULL CHECK	NULL means ongoing as then known
is_withdrawn	boolean	NN D=false	Retracts an incorrect episode

FK (candidate_id) → party(party_id); FK (predecessor_id) → work_experience_event(experience_event_id).

Checks: btrim(employer_name) <> ''; btrim(job_title) <> ''; end_date IS NULL OR end_date >= start_date; predecessor_id IS NULL OR predecessor_id <> experience_event_id.

Example data

experience_event_id	predecessor_id	employer_name	end_date
600	NULL	Old Harbor Retail	NULL
601	600	Old Harbor Retail	2026-01-31

location_preference_event

Separate rows allow several preferred places and later withdrawals. The event key determines the candidate, label, state and times; there is no repeated geographical metadata. The self-FK preserves one preference history, and a trigger prevents duplicate candidate/location roots.

Field name	PostgreSQL type	Constraints	Field meaning
preference_event_id	bigint	PK IDENTITY	Preference assertion identifier
candidate_id	bigint	FK NN	Person expressing the preference
predecessor_id	bigint	FK UQ NULL	Earlier version of this preference
effective_at	timestamptz	NN	Business validity start
recorded_at	timestamptz	NN D=now()	System recording time
location_label	text	NN CHECK	One canonical place label or remote
is_preferred	boolean	NN D=true	False withdraws the preference

FK (candidate_id) → party(party_id); FK (predecessor_id) → location_preference_event(preference_event_id).

Checks: btrim(location_label) <> ''; predecessor_id IS NULL OR predecessor_id <> preference_event_id.

Example data

preference_event_id	predecessor_id	location_label	is_preferred
700	NULL	Vilnius LT	true
701	700	Vilnius LT	false

candidate_service_event

Each root is one service session, independent of job applications; successors record rescheduling, completion or cancellation. The event key determines the snapshot. service_type does not determine fee_eur because fees are agreed per session; no catalog price or description is duplicated.

Field name	PostgreSQL type	Constraints	Field meaning
service_event_id	bigint	PK IDENTITY	Service session assertion identifier
candidate_id	bigint	FK NN	Person receiving the service
predecessor_id	bigint	FK UQ NULL	Earlier version of this session
effective_at	timestamptz	NN	When this snapshot takes effect
recorded_at	timestamptz	NN D=now()	System recording time
service_type	text	NN CHECK	resume_advice coaching skills_testing
scheduled_at	timestamptz	NN	Session appointment time
status	text	NN D=scheduled CHECK	scheduled completed cancelled
fee_eur	numeric(10,2)	NN D=0 CHECK	Charge for this session in EUR
result_note	text	NULL	Advice or test result summary

FK (candidate_id) → party(party_id); FK (predecessor_id) → candidate_service_event(service_event_id).

Checks: service_type IN ('resume_advice','coaching','skills_testing'); status IN ('scheduled','completed','cancelled'); fee_eur >= 0; result_note IS NULL OR status = 'completed'; predecessor_id IS NULL OR predecessor_id <> service_event_id.

Example data

service_event_id	predecessor_id	service_type	status
800	NULL	coaching	scheduled
801	800	coaching	completed

Extension and Validation Notes

Adding profile attributes or more people requires no change to the recruitment flow. Panel interviews would require an interview_participant bridge; configurable skills, locations or services would justify catalogs. Repeat submissions would require an application attempt key. These are the first likely extensions, rather than extra repeating columns.

Checks performed: 15 tables and 21 FK mappings; connected sample references and keys; negative cases for duplicate applications, missing job versions, invalid participant roles, competing successors, wrong chain ownership, duplicate placements and inverted interview times. These are structural checks, not PostgreSQL execution. Composite revision keys and self-references provide the optional advanced features.

Business source: supplied Recruitment Agency description. Structure: supplied Business_Template.docx. PostgreSQL references: postgresql.org/docs/current/ddl-constraints.html and postgresql.org/docs/current/datatype-datetime.html. Cross-row enforcement remains part of the later DDL task.